RAHUL SHETTY ACADEMY

API TESTING INTERVIEW QUESTIONS

50 commonly asked questions with concise, interview-ready answers

QA / SDET / AUTOMATION ENGINEER

rahulshettyacademy.com

API Fundamentals

1. What is an API?

An API (Application Programming Interface) allows two software systems to communicate through defined requests and responses. For example, an e-commerce application can call a payment gateway API to authorize a transaction.

2. What is API testing?

API testing verifies an API's functionality, reliability, performance, security, and data accuracy by sending requests directly to endpoints and validating the responses. It tests the service layer without interacting with browser elements.

3. Why is API testing important?

It detects defects earlier, executes faster than UI testing, validates core business logic, gives more stable automation, and identifies integration, data, security, and performance issues.

4. What do you validate in an API response?

Validate the status code, response body, headers, schema, response time, data types, business rules, error details, authorization behavior, and relevant database or downstream-system changes.

5. What is the difference between REST and SOAP?

REST is an architectural style that is lightweight and commonly exchanges JSON over HTTP. SOAP is a protocol, uses XML envelopes, is more structured, and supports standards such as WS-Security. REST can support multiple representations; SOAP messages are XML-based.

6. What is a RESTful API?

A RESTful API models data as resources identified by URLs, uses standard HTTP methods, keeps communication stateless, separates client and server responsibilities, and returns consistent resource representations.

```
GET /customers/101
```

7. What does statelessness mean in REST?

Every request contains all information needed for processing. The server does not depend on conversational state saved from a previous request; a protected request normally includes its authentication token each time.

HTTP Methods and Status Codes

8. Explain the commonly used HTTP methods.

GET retrieves a resource; POST creates a resource or initiates an operation; PUT replaces or fully updates; PATCH partially updates; DELETE removes; HEAD retrieves headers without a body; OPTIONS reports supported communication options.

9. What is the difference between PUT and PATCH?

PUT generally sends a complete replacement representation, while PATCH changes selected fields. Always confirm the exact behavior from the API contract.

```
PUT /users/10
PATCH /users/10
```

10. What is an idempotent HTTP method?

Repeating an idempotent request has the same intended effect on server state as sending it once. GET, PUT, and DELETE are designed to be idempotent. POST generally is not, because repeats may create multiple resources.

11. What is the difference between POST and PUT?

POST normally creates a resource under a collection and lets the server assign its identifier. PUT creates or completely replaces a resource at a known URI.

```
POST /users
PUT /users/101
```

12. Explain the HTTP status-code categories.

1xx is informational, 2xx success, 3xx redirection, 4xx client error, and 5xx server error.

13. What is the difference between 200, 201, 202, and 204?

200 means success; 201 means a resource was created; 202 means accepted for processing, often asynchronously; 204 means success with no response body.

14. What is the difference between 400, 401, 403, and 404?

400 is an invalid request; 401 means authentication is missing or invalid; 403 means the caller is authenticated but not permitted; 404 means the resource was not found or is intentionally hidden.

15. What is the difference between 409 and 422?

409 indicates a conflict with current resource state, such as a duplicate. 422 means the syntax is valid but the content violates semantic validation or a business rule.

16. What is the difference between 500, 502, 503, and 504?

500 is an unexpected server failure; 502 is an invalid upstream response; 503 means the service is temporarily unavailable; 504 means an upstream service did not respond in time.

Requests, Headers, and Data

17. What are the main components of an API request?

Base URL, resource path, HTTP method, path parameters, query parameters, headers, cookies, authentication information, and an optional request body.

18. What is the difference between path and query parameters?

Path parameters identify a resource, such as /users/101. Query parameters filter, sort, search, or paginate, such as /users?status=active&page;=2.

```
GET /users/101
GET /users?status=active&page=2
```

19. What is the difference between Content-Type and Accept?

Content-Type describes the format of the body being sent. Accept tells the server which response representations the client can process.

```
Content-Type: application/json
Accept: application/json
```

20. What are serialization and deserialization?

Serialization converts an application object into JSON, XML, or another transferable format. Deserialization converts a response representation back into a programming-language object.

21. What is JSON schema validation?

It validates response structure: required properties, data types, nested objects, arrays, allowed values, and constraints. It complements but does not replace assertions on business values.

22. How do you test API pagination?

Test default and custom page sizes, first/middle/last pages, out-of-range pages, total counts, duplicate or missing records between pages, invalid inputs, maximum size, cursors or navigation links, and stable ordering.

23. How do you test sorting and filtering?

Cover supported fields, ascending and descending order, single and combined filters, exact and partial matches, case rules, special characters, invalid fields or values, empty results, and interaction with pagination.

Authentication and Security

24. What authentication mechanisms have you tested?

Common mechanisms include Basic authentication, API keys, Bearer tokens, OAuth 2.0, JWTs, session cookies, and client certificates.

25. What is the difference between authentication and authorization?

Authentication proves who the caller is. Authorization determines what that caller is permitted to access or change.

26. What is OAuth 2.0?

OAuth 2.0 is an authorization framework that gives clients limited access to protected resources without sharing a user's password. Common flows include Authorization Code with PKCE, Client Credentials, and Refresh Token.

27. What is a JWT?

A JSON Web Token normally contains Header.Payload.Signature. The header identifies the algorithm, the payload carries claims, and the signature detects tampering. The payload is encoded, not inherently encrypted.

```
eyJhbGciOi... . eyJzdWIiOi... . SflKxwRJS...
```

28. How would you test token-based authentication?

Test valid, missing, malformed, expired, not-yet-valid, modified, revoked, wrong-issuer, wrong-audience, and insufficient-scope tokens. Verify refresh, logout, and revocation behavior where supported.

29. What API security scenarios should an SDET understand?

Broken object and function authorization, broken authentication, unrestricted resource consumption, property-level authorization, injection, security misconfiguration, poor inventory management, unsafe third-party API consumption, and abuse of sensitive business flows.

30. How would you test broken object-level authorization?

Authenticate as User A, request User A's resource, then change the identifier to one owned by User B without changing the token. The API must reject access and must not expose User B's data.

API Automation

31. How do you design an API automation framework?

Use an API client or request-specification layer, service classes, request/response models, test-data builders, authentication utilities, environment configuration, schema files, reusable assertions, logging, reporting, tags, and CI/CD integration.

32. What is a request specification in REST Assured?

It stores reusable configuration such as the base URI, base path, headers, authentication, content type, and logging rules. Tests reuse it instead of duplicating setup.

```
RequestSpecification spec = new RequestSpecBuilder()
    .setBaseUri("https://api.example.com")
    .setContentType(ContentType.JSON)
    .build();
```

33. How do you validate a response using REST Assured?

Build the request in given(), send it in when(), and validate status, headers, content type, and body in then(). Keep common setup in specifications and domain-level assertions in reusable helpers.

```
given().spec(spec).pathParam("id", 101)
.when().get("/users/{id}")
.then().statusCode(200)
.body("status", equalTo("active"));
```

34. How do you use response data in another request?

Extract a generated value such as an ID or token from the first response and use it in a later GET, update, or DELETE request. This is commonly called request chaining.

```
String id = given().body(payload)
    .when().post("/users")
    .then().statusCode(201)
    .extract().path("id");
```

35. How do you write an API test in Postman?

Add JavaScript assertions in the post-response script. Validate status, headers, timing, schema, and body values using pm.test and pm.expect.

```
pm.test("Status code is 200", () => {
  pm.response.to.have.status(200);
});
pm.expect(pm.response.json().status).to.eql("active");
```

36. How do you manage test data?

Create data through APIs, generate unique values, use builders or factories, keep tests independent, use dedicated accounts, clean up after execution, and never hardcode sensitive credentials.

37. How do you manage tokens in a framework?

Centralize token creation, cache tokens while valid, check expiration, refresh when required, keep secrets outside source control, and mask credentials and tokens in logs.

38. How do you prevent API tests from becoming flaky?

Avoid execution-order dependencies, use independent data, poll instead of fixed sleeps, control external dependencies, configure realistic timeouts, avoid exact assertions on dynamic fields, clean up data, and capture diagnostics.

39. Should API automation validate the database?

Yes when access is allowed and end-to-end data integrity matters, but do it selectively. Excessive database assertions tightly couple tests to implementation details; the public API contract should remain the primary behavior under test.

Scenario-Based Interview Questions

40. How would you test a Create User API?

Cover mandatory-only and full valid payloads; null, empty, duplicate, invalid-format, invalid-type, boundary, unauthorized, and unsupported-field cases. Validate status, schema, ID creation, retrieval, persistence, response time, and that sensitive data is not returned.

41. How would you test a DELETE API?

Test existing and missing resources, repeated deletion, missing authorization, another user's resource, dependency conflicts, soft versus hard deletion, audit updates, and a follow-up GET.

42. An API returns 200, but the test should fail. Why?

HTTP success does not prove business correctness. The body may contain incorrect data or an embedded error; required fields can be missing; schema, security, performance, persistence, or business-rule validation may fail.

43. How do you test an asynchronous API?

Validate the initial 202 response, extract the job or correlation ID, poll at controlled intervals, enforce a maximum timeout, stop on a terminal state, and validate final output, errors, duplicates, and side effects.

44. How would you test API rate limiting?

Send calls up to and beyond the threshold. Verify successful calls within the limit, 429 beyond it, rate-limit and Retry-After headers, reset behavior, scope per key/user/IP/tenant, and recovery after the waiting period.

45. How do you test retry behavior?

Verify only transient failures are retried, the maximum attempts, delay or exponential backoff, total timeout, no retry for permanent client errors, logging, and idempotency so retries do not create duplicates.

46. What is contract testing?

It verifies that a provider and its consumers follow an agreed endpoint, method, payload, field, type, and status-code contract. Consumer-driven tools such as Pact can catch breaking changes without a full end-to-end environment.

47. What is API mocking?

A mock simulates a dependency that is unavailable, costly, slow, unstable, unfinished, or hard to force into error states. Mock-based tests should be complemented with integration tests against the real service.

48. How do you test API backward compatibility?

Confirm existing fields and types remain stable, previously valid requests still work, additions are safely optional, older clients tolerate new fields, status behavior stays consistent, and deprecation/versioning rules are honored.

49. What logs would you capture when an API test fails?

Capture method, endpoint, sanitized headers, request and response bodies, status, response time, timestamp, environment, test-data IDs, and correlation or trace IDs. Mask credentials, tokens, keys, and personal data.

50. What is your overall API-testing strategy?

Understand the contract and business purpose first. Cover positive, negative, boundary, authorization, integrity, idempotency, contract, performance, and security risks. Automate stable regression checks and run them at suitable CI/CD stages with useful diagnostics.

Interview tip	Do not memorize answers word for word. Connect each concept to an API, defect, framework decision, or debugging experience from your own project.